

A FINAL PROJECT
REPORT ON

**Decentralized Aggregation Mechanism for Training
Machine Learning models using Smart Contract
System**

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY,
PUNE IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

BACHELOR OF ENGINEERING
Computer Engineering

BY

Akash Jaju	Exam No:	B400050423
Sumeet Mali	Exam No:	B400050490
Pradip Funde	Exam No:	B400050389

Under the guidance of
**Prof. M.V.Mane &
Prof . Deepika Kumari**



DEPARTMENT OF COMPUTER ENGINEERING
PUNE INSTITUTE OF COMPUTER TECHNOLOGY
PUNE - 411 043.
2024 -2025

SCTR's PUNE INSTITUTE OF COMPUTER TECHNOLOGY
DEPARTMENT OF COMPUTER TECHNOLOGY



C E R T I F I C A T E

This is to certify that the final project report entitled
**Decentralized Aggregation Mechanism for Training Machine Learning
models using Smart Contract System**
submitted by

Akash Jaju	Exam No: B400050423
Sumeet Mali	Exam No: B400050490
Pradip Funde	Exam No: B400050389

is a bonafide work carried out by them under the supervision of **Prof. M.V.Mane & Prof Deepika Kumari** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University for the award of the Degree of Bachelor of Engineering (Computer Engineering).

This project report has not been earlier submitted to any other Institute or University for the award of any degree or diploma.

Prof. M.V.Mane
Project Guide

Dr. G.V .Kale
HOD CE

Prof Deepika Kumari
Co -Guide

Dr. S. T. Gandhe
Principal

Date:
Place:

ACKNOWLEDGEMENT

I would like to express my profound appreciation to Professor M.V.Mane for his expert guidance, and Professors Deepika Kumari for their meticulous review of my final year project, "Decentralized Aggregation Mechanism for Training Machine Learning Models using Smart Contract System." Additionally, I am grateful to Dr.G.P.Potdar for his invaluable coordination and administrative support as the overall coordinator. Your collective contributions in expertise , have been instrumental in the successful completion of this endeavor, and I am sincerely thankful for your unwavering support and dedication to this project.

Akash Jaju

Sumeet Mali

Pradip Funde

ABSTRACT

Data privacy and sharing have always been pressing challenges in the development of complex deep learning-based systems for modeling various types of data. Researchers have actively explored decentralized approaches that leverage data from multiple nodes without the need to physically merge their contents. In this paper, we propose a solution to harness the benefits of a distributed data setup in training deep learning architectures, leveraging a smart contract system. Our proposed mechanism focuses on aggregating intermediate representations obtained from local Artificial Neural Network (ANN) models using a blockchain. The training of these local models occurs on their respective data, and the resulting intermediate representations, when combined and trained together on a host node, yield a more accurate system. While federated learning typically addresses scenarios where the same data features are distributed across multiple nodes, our approach deals with situations where the same number of data samples exist, but their features are distributed across various nodes. This methodology extends beyond specific use cases such as bank loan prediction. To illustrate the versatility of our approach, We consider the task of bank loan prediction, where personal and bank-specific details of individuals may not be available in the same location. Our aggregation mechanism allows the training of a model on distributed data without the need to share or concatenate the actual data values. The results demonstrate that our approach consistently outperforms individual nodes and performs on par with centralized data setups, establishing a strong case for its application across diverse architectures and tasks. This solution has broad implications for organizations seeking to train deep learning models on vertically partitioned data in various domains beyond the realm of bank loans.

Keywords: Data Privacy, Blockchain, Distributed Data, Smart Contract, Federated Learning, Aggregation Mechanism

TABLE OF CONTENTS

Certificate	i
Acknowledgement	ii
Abstract	iii
Contents	v
List of Figures	vii
List of Tables	viii
Abbreviations	ix
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Objectives	1
1.4 Scope	1
2 Literature Survey	2
2.1 Existing Methodologies	2
2.2 Research Gap Analysis	3
3 Requirement Specification and Analysis	7
3.1 Problem Definition	7
3.2 Scope	7
3.3 Objectives	7
3.4 Proposed Methodology	7
3.4.1 Smart Contract Design:	8
3.4.2 Data Encryption and Privacy Preservation:	8
3.4.3 Federated Learning Setup:	8
3.4.4 Secure Aggregation Protocol:	8
3.4.5 Incentive Mechanism:	8
3.4.6 Blockchain	8
3.5 Project Requirements	11
3.5.1 Datasets	11
3.5.2 Functional Requirements	12
3.5.3 Non Functional Requirements	12

3.5.4	Hardware Requirements	13
3.5.5	Software Requirements.....	13
3.5.6	Project Resources.....	15
3.5.7	Module Split-up.....	15
3.5.8	Functional Decomposition	16
3.5.9	Project Team Role and Responsibilities	16
3.5.10	Project Plan 3.0.....	17
3.5.11	Conclusion.....	18
3.5.12	PERT Table.....	18
3.5.13	PERT Diagram.....	19
4	System Analysis and Design	20
4.1	System Architecture	20
4.2	Necessary UML Diagrams.....	21
4.2.1	DFD	21
4.2.2	Activity Diagram	22
4.2.3	Sequence Diagram	23
4.3	Algorithm and Methodologies.....	23
5	Implementation	26
5.1	Stages of Implementation.....	26
5.1.1	Data Preprocessing.....	26
5.1.2	Implementation of Modules.....	26
5.2	Experimentation Setup.....	27
6	Results	28
6.1	Results of Experiments	28
6.2	Result Analysis	28
6.2.1	Experiments	28
6.3	Testing	30
7	Conclusion and Future Scope	32
7.1	Conclusion.....	32
7.2	Limitations of the Project	32
7.3	Future Scope	33
	References	35

List of Figures

3.1	Representing the model as bytes in Python	9
3.2	FL Data in Solidity	10
3.3	Training Stages in Sol.....	10
3.4	PERT Diagram Of Project	19
4.1	System Architecture	20
4.2	Data Flow Diagram	21
4.3	Activity Diagram	22
4.4	Sequence Diagram	23
6.1	Accuracy vs. Epoch Plots with Varying C_{pre}	29

List of Tables

2.1 Literature Survey	6
3.1 Revised Project Timeline.....	17
3.2 PERT Table	18
6.1 Byte Size vs. Gas Fee.....	31

Abbreviations

MANET	:	Mobile Ad-Hoc Network
LMST	:	Local Minimum Spanning Tree

1. Introduction

1.1 Introduction

Data privacy and sharing have always been pressing challenges in the development of complex deep learning-based systems for modeling various types of data. Researchers have actively explored decentralized approaches that leverage data from multiple nodes without the need to physically merge their contents. In this paper, we propose a solution to harness the benefits of a distributed data setup in training deep learning architectures, leveraging a smart contract system.

1.2 Motivation

Address data privacy and sharing challenges in deep learning by facilitating a decentralized approach that leverages distributed data without physically merging it. This is essential for building complex deep learning models while preserving data privacy.

1.3 Objectives

- ' Develop a mechanism that aggregates intermediate representations from local ANN models over a blockchain.
- Enable model training on locally held data without sharing or concatenating actual data values.
- ' Improve model accuracy by combining and training intermediate representations on a host node.
- Validate the approach's effectiveness through bank loan prediction and demonstrate superior performance compared to individual nodes.

1.4 Scope

This solution is applicable to organizations dealing with vertically partitioned data and aims to revolutionize deep learning model training by preserving data privacy while achieving centralized data setup performance.

2. Literature Survey

2.1 Existing Methodologies

By utilising blockchain technology and smart contracts, these techniques seek to enable co-operative machine learning model training among a decentralised network of participants while guaranteeing data privacy, security, and trust. Here are a few current approaches and methods used in this field:

- ' Federated Learning with Smart Contracts
- Secure Multi-Party Computation (MPC)
- Homomorphic Encryption
- Token-based Incentive Mechanisms
- Consensus Mechanisms
- ' Permissioned Blockchains
- ' Decentralized Identity and Reputation Systems
- ' Off-Chain Compute and On-Chain Verification
- zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge)

These are various approaches defined in various research papers and these models show different use cases depending upon data types and privacy and security.

As our problem can also be solved using the traditional way but with less or no security on a centralized repository with different models. These approaches are particular to use cases, data kinds, and security and privacy levels. Although they provide a simpler option, traditional centralised repositories can jeopardise data privacy and security.

Centralised models are a good option, nevertheless, in some situations where a convenience trade-off may be acceptable. However, in order to make well-informed judgements on data management, one must have a thorough grasp of the various techniques and their applicability. This will guarantee that the model selected will meet the particular needs of a certain application or problem.

2.2 Research Gap Analysis

Table 2.1: Literature Survey

Sr. No	Sources	Techniques used	Granularity	Evaluation
1	[1]	In addition to focusing on scaling federated learning for larger applications, the research attempts to increase the efficiency of federated learning.	The precise conclusion of the study is not provided in the snippet. On the other hand, it is possible to deduce from the information provided that the authors believe federated averaging to be a useful method for federated learning.	The study may examine the effects of making particular assumptions about network connection. The article may examine the potential gap of assuming hardware homogeneity.
2	[2]	Strategies to improve federated learning's communication efficiency are examined in this research. Additionally, the paper addresses trade-offs between privacy and model quantization or compression.	The paper ends with some observations or suggestions for enhancing federated learning communication effectiveness. It might also emphasise how crucial it is to take privacy trade-offs into account when putting model quantization and compression techniques into practise.	The study examines the shortcomings and restrictions of current model compression and quantization methods, particularly as they relate to federated learning.
3	[3]	The phrase "Differential Privacy," refers to a particular framework for calculating the privacy assurances offered by data processing methods.	The study ends with some observations or suggestions about data privacy techniques and the privacy vs. performance trade-off.	The study examines the shortcomings and difficulties in enhancing the effectiveness of privacy-preserving techniques such as safe multiparty computation and differential privacy.

Sr. No	Sources	Techniques used	Granularity	Evaluation
4	[4]	Model aggregation techniques are the main tools used in this paper. implying that textual data processing and aggregation may require some kind of natural language processing (NLP) method.	the paper may not address large-scale, image-based scenarios but concentrates on smaller, text-based use cases.	It states that the model was created using a little dataset and that an assessment of the model's functionality, precision, or ability to handle brief text descriptions may be included.
5	[5]	The paper investigates methods for federation optimisation. Hardware optimisation and system architecture are especially mentioned.	This covers system settings, hardware parts, and architectural design. The specifics of how these optimisations can be made may be covered in fully in the article.	It also discusses the implications of these optimisations for regulatory compliance and system interoperability.
6	[6]	The study examines adversarial robustness and homomorphic encryption methods.	It concentrate on the parts of adversarial robustness and homomorphic encryption that are applicable in real-world settings. It goes into great detail on how these methods are applied in actual situations and how they comply with homomorphic encryption regulations.	The evaluation includes a realistic and efficacious assessment of the adversarial robustness and homomorphic encryption solutions.

Sr. No	Sources	Techniques used	Granularity	Evaluation
7	[7]	The study investigates decentralisation and blockchain approaches.	The specifics of how these technologies might be applied to enhance data security and transparency in diverse applications may be covered in length in the paper.	The effectiveness and environmental impact of blockchain systems based on PoW are discussed. It might also investigate different consensus processes and the energy trade-offs associated with them.
8	[8]	According to the study, the main focus is on quality evaluation, which includes assessing the calibre and dependability of previous research, data, or evidence and may entail systematic review techniques.	The methods, information, and findings of earlier research are analysed and critiqued in this work.	Since this review is systematic, publication bias may exist.
9	[9]	In research, quality assessment usually entails assessing the calibre and dependability of previous investigations, data, or supporting documentation.	It attempts to evaluate the calibre of previous studies and deal with the particulars of research gaps in the area.	The goal of systematic reviews is to present an objective and thorough synthesis of the body of knowledge. Insufficient past data paired with elevated energy usage.

Sr. No	Sources	Techniques used	Granularity	Evaluation
10	[10]	This study suggests enhancing the present transaction process with a Blockchain-based infrastructure. The idea of tokenization is introduced for transactions as well.	All documents in the existing system are in hard copy, which is a pain issue. If papers could be kept on blockchain, data integrity and data linearity would be ensured.	The creation of a blockchain application that may enhance the authentication transaction process will be the main topic of this paper.
11	[11]	This document clarifies data ownership, IoT integration, and immutable record keeping.	The technical and practical issues of employing blockchain technology for data sharing within a closed network are the main emphasis of this study.	This article assesses the problems of operational efficiency and integration while evaluating the overall success of blockchain use.
12	[12]	This article discusses the opportunity that blockchain technology can offer to address the different issues that the existing traditional system faces.	This paper analyses a brief on startups and organisations that have addressed that issue and how they did it.	This document outlines the issues with the current system that need to be addressed. So after reading this paper and resolving the issues raised, it's a nice place to start for implementation.

3. Requirement Specification and Analysis

3.1 Problem Definition

The challenge is to create a system that allows multiple parties to collaborate in training machine learning models using blockchain technology. This involves developing secure data sharing and aggregation protocols, integrating smart contracts for managing the process, ensuring data privacy, and designing efficient incentive mechanisms. The aim is to build a scalable and fair system that guarantees the security and integrity of the shared data while incentivizing active participation from all parties involved.

3.2 Scope

This solution is applicable to organizations dealing with vertically partitioned data and aims to revolutionize deep learning model training by preserving data privacy while achieving centralized data setup performance.

3.3 Objectives

- Establishing a secure and transparent platform for collaborative model training.
- Ensuring data privacy and confidentiality during the aggregation process.
- Creating an efficient and scalable network infrastructure for handling a large number of participants and data.
- Implementing a fair incentive mechanism to encourage active participation and contributions.

Developing a robust smart contract system for managing the aggregation process effectively and securely.

3.4 Proposed Methodology

Implementing a decentralized aggregation mechanism for training machine learning models using a smart contract system involves combining principles from blockchain technology, cryptography, and federated learning. Below is a proposed methodology for this purpose:

341 Smart Contract Design:

Define the structure of the smart contract, including data storage, permission management, and reward distribution. Specify the conditions for triggering the aggregation process, such as reaching a certain number of model updates or a specific time interval.

342 Data Encryption and Privacy Preservation:

Employ homomorphic encryption or multi-party computation to allow participants to compute on encrypted data without revealing the actual data. Ensure that sensitive data remains private throughout the aggregation process.

343 Federated Learning Setup:

Establish a secure communication channel between the participating devices and the decentralized network. Distribute the initial model to the participants, who will train the model locally using their own data.

344 Secure Aggregation Protocol:

Utilize cryptographic techniques, such as secure multi-party computation (MPC), to securely aggregate the model updates from different participants. Design a consensus mechanism to verify the validity of the aggregated updates before incorporating them into the global model.

345 Incentive Mechanism:

Integrate a token-based reward system to incentivize participants for contributing their computational resources and data. Implement a fair and transparent reward distribution mechanism based on the quality and quantity of contributions.

3.4.g Blockchain

Our primary blockchain platform of choice is Ethereum, given its widespread usage and robust tooling support. Consequently, we opted for the Solidity programming language for developing the smart contract.

For interaction with the Ethereum blockchain, the 'ethers' library offers a streamlined Python interface. Due to the high cost of running this project on the actual Ethereum network, we utilized the 'eth-tester' tool suite for project execution.

Moreover, we structured this blockchain platform in a modular fashion. Within the code, the `BlockchainModule` module manages communication with the Ethereum ecosystem, and its flexible design enables easy replacement. For instance, we provide a `MockBlockchainModule` module capable of emulating the functions of the previous module without interfacing with a real blockchain. This feature allows us to circumvent the runtime performance overhead of blockchain. Additionally, this module proves useful for users interested in testing only the federated learning component without the need to install any blockchain libraries.

In Solidity, the model state is represented as bytes, achieved by aggregating all model parameters in IEEE 754 floating point format and obtaining the corresponding byte representation. Subsequently, this byte data can be utilized by interpreting the byte buffer as an array with the appropriate data type and shape. Figure 3.1 illustrates this process in Python.

```
# Model to bytes
bytestr = b''
for param in model1.parameters():
    bytestr += param.detach().numpy().tobytes()

for param in model2.parameters():
    arr = param.detach().numpy()
    arr[:] = np.frombuffer(bytestr[:arr.nbytes], dtype=arr.dtype).reshape(arr.shape)
    bytestr = bytestr[arr.nbytes:]
```

figure 3.1: Representing the model as bytes in Python

The global model is stored as a property in the contract, as well as mean and standard deviation data. The local counterparts of these are handled by events in Solidity (Figure 3.2, which are more appropriate for this use case. This design decision also reduces the gas fee paid by the clients.

An alternative option is to enable clients to transmit their local updates to the server outside the blockchain, which may be preferred by some clients due to the limited privacy provided by Ethereum and most open blockchains in general (Tikhomirov, 2017). This setup allows for a combination of clients communicating through the blockchain and others communicating externally. One advantage of an all-blockchain approach is that it enables verification by others to ensure the correct execution of model averaging by the server.

It is crucial to ensure that only the server, specifically the contract owner in Solidity, executes the global updates. For this purpose, a modifier named `OwnerOnly` is imple-

```

1  // Representation of the global model in hylcs.
2  bytes public code1 :
3  // For data standardization, global means on strls.
4  bytes public means :
5  bytes public stds;
6
7  // This event is fired when a client reports its local update.
8  event LocalUpdate(address indexed from, uint epoch,
9                    uint size, bytes model);
10 // These events are provided to the preprocessor in the server.
11 // by the clients.
12 event LocalMeans(address indexed from, uint size, bytes data);
13 event LocalStd(address indexed from, uint size, bytes data);

```

Figure 3.2: FL Data in Solidity

mented. Relevant portions of the code are depicted in Figure 77. Although this setup may appear to create a single point of failure, transactions being on the blockchain allow anyone to retrieve the latest or a previous version of the model and create a new contract.

```

1  contract FederatedLearningContract {
2      address public owner
3      constructor() public {
4          owner = msg.sender;
5          // ...
6      }
7
8      // With this modifier, only the owner can call the given function.
9      modifier OwnerOnly() {
10         require(msg.sender == owner, "Only the contract owner can call
11             this function!");
12     }
13
14     // that public modifier in Solidity does not limit the access
15     // to the owner.
16     function globalUpdate(bytes memory updateModel) public OwnerOnly
17     {
18         // ...
19     }
20
21 }

```

88 \end{minted}

20 & \caption{Owner Only modifier in Solidity}

21 \$ \label{fig:owneronly}

Enums are employed to monitor the training stage, as illustrated in Figure 3.3. The server coordinates the training process by transmitting global mean/standard deviation data and updates.

Figure 3.3: Training Stages in Sol

For simplicity, the focus was solely on implementing the learning system on the

blockchain, overlooking other potential transactions between clients and the server. However, incorporating additional features is straightforward given the foundational blockchain-based federated learning system already in place. For example, the server could compensate clients for providing local updates and local ϵ and δ reports, addressing privacy concerns and offering a financial incentive to encourage greater participation in the training process.

3.5 Project Requirements

3.5.1 Datasets

Network Intrusion Detection Dataset

The dataset provided for auditing encompasses a broad spectrum of intrusions simulated within a military network environment. This simulation aimed to capture raw TCP/IP dump data by emulating a typical LAN setup akin to that of the US Air Force. The LAN environment was meticulously crafted to closely resemble real-world conditions and subjected to various attack scenarios. In this context, a connection is defined as a sequence of TCP packets initiating and concluding within a specified time frame, facilitating the exchange of data between a source and target IP address following well-defined protocols. Each connection is categorized as either normal or indicative of an attack, with precise identification of the attack type. Each connection record comprises approximately 100 bytes of data. From each TCP/IP connection, 41 features, comprising both quantitative and qualitative attributes, are derived from normal and attack data sets, including 3 qualitative and 38 quantitative characteristics. The class variable encompasses two distinct categories: 1) Normal 2) Anomalous.

Credit Dataset

The bank's consumer lending division wants to automate the decision-making process regarding mortgage loan approval. In doing so, they will follow the recommendations of the Equal Credit Opportunity Act and set an example, including modeling a good credit score. The model will be based on information collected from applicants who have recently received loans through the existing credit system. The model can be improved using predictive modeling tools, but the model must be interpretable enough to provide justification for any negative (rejection) errors.

Machine Predictive Maintenance Classification

Predictive Maintenance Classification Since real-life maintenance information is often difficult to obtain and especially difficult to publish, we present and provide detailed information based on our own experience, showing what is experienced in real business maintenance.

Income Dataset

This data includes education, employment, marital status, etc. to estimate whether the salary is higher than 50,000. It provides an estimate of properties such as. Can be used to apply specified training problems such as distribution.

352 Functional Requirements

- Data Aggregation
- Blockchain Integration
- ' Smart Contract Management
- ' Decentralized Training
- ' Performance Benchmarking
- ' Data Privacy
- ' Compatibility
- ' Security

353 Non Functional Requirements

- Data Privacy and Security
- Scalability
- ' Performance
- Reliability
- ' Transparency
- ' Compliance
- ' Interoperability

- Documentation
- ' Scalable Infrastructure

354 Hardware Requirements

GPU Support: Our machine learning framework is configured to leverage GPU acceleration. This may require GPU drivers and CUDA support.

Memory (RAM): For deep learning tasks, a minimum of 16 GB is recommended.

Server/Cloud Infrastructure: Servers or cloud instances to host the blockchain network and smart contract system. High-performance servers or cloud instances to handle the computation-intensive tasks of machine learning model training. Scalable infrastructure to accommodate the growing network of participants and data.

Blockchain Nodes: Nodes for participating in the blockchain network. These nodes are essential for maintaining the blockchain's distributed ledger and executing smart contracts.

Storage: Sufficient storage capacity for storing blockchain data, smart contracts, and the datasets used in machine learning. Distributed file systems or network-attached storage (NAS) for efficient data sharing.

Networking Equipment: High-speed internet connections for efficient data transfer between participants and the blockchain network. Routers, switches, and other networking equipment to ensure a stable and secure network environment.

Security Hardware: Hardware security modules (HSMs) for securing cryptographic keys and ensuring the integrity of smart contracts and data. Firewalls and intrusion detection/prevention systems to protect the network from security threats.

Blockchain-Specific Hardware: In some cases, specialized hardware, such as Application-Specific Integrated Circuits (ASICs) for certain blockchain platforms, may be required for mining or validation.

Backup and Redundancy: Redundant hardware to ensure high availability and data backup solutions to prevent data loss.

355 Software Requirements

Blockchain Infrastructure: The system must operate on a blockchain infrastructure to facilitate the secure exchange and aggregation of data from multiple nodes.

- ' Solidity
- ' Ether.js

' IPFS

Machine **Learning** Framework: Need of a machine learning framework or library, and the training of local Artificial Neural Network (ANN) models on individual nodes, making use of their respective data.

' TensorFlow

- PyTorch

- Other standard ML python libraries to implement and train deep learning models.

Version Control: A version control system (e.g., Git) to manage and track changes in code and models.

Integrated Development Environment (IDE): You will need an IDE or code editor to write, debug, and manage your code. Popular choices includes

- Visual Studio Code

' PyCharm

' Jupyter Notebook (for data science and machine learning)

Database Management System: Depending on your application, you might need a database system to store and manage non-blockchain-related data. Popular choices includes

- PostgreSQL

' MySQL or NoSQL

- MongoDB

Web Framework (**Optional**): If your system involves a web application, you may require a web framework like

' Django

- Flask or Node.js

to build the front-end and backend components for user interactions.

Monitoring and Logging Tools (Optional): Implement monitoring and logging solutions like

- ' Prometheus
- ELK Stack (Elasticsearch, Logstash, Kibana)

or others to keep track of system performance and troubleshoot issues.

3.5.g Project Resources

The successful completion of the project on "Decentralized Aggregation Mechanism for training Machine Learning models using "Smart Contract System" the following resources are essential "

3.5.7 Module Split-up

1. Decentralized Architecture Design:

Define the structure for decentralized data storage and processing. Establish protocols for peer-to-peer communication and interaction.

2. Smart Contract Implementation:

Develop smart contracts for managing data access and permissions. Design functionalities for triggering and managing model aggregation.

3. Data Privacy and Security-

Implement encryption techniques for secure data transmission. Ensure privacy preservation during the aggregation process.

4. Federated Learning Framework:

Set up protocols for distributed model training across multiple nodes. Establish communication channels for exchanging model updates securely.

5. Aggregation Protocol Development:

Employ cryptographic methods for secure aggregation of model updates. Integrate consensus algorithms for validating aggregated results.

6. Incentive Mechanism Integration:

Create a token-based reward system for encouraging participation. Design algorithms for fair and transparent reward distribution.

7. Governance and Compliance Framework:

Develop protocols for managing the decentralized network. Ensure compliance with relevant regulatory requirements and data protection standards.

8. Testing and Security Assurance:

Conduct comprehensive testing of the entire system for robustness and security. Conduct vulnerability assessments and audits to find and reduce possible risks.

3.5.8 Functional Decomposition

The project's functional decomposition outlines the major functions and sub-functions that contribute to its successful execution. It includes, data preprocessing, model evaluation, and documentation.

3.5.9 Project Team Role and Responsibilities

To ensure a smooth project execution, the following team roles and responsibilities are defined:

Machine Learning - Om Acharya, Advait Naik

- Selection of Model
- Model evaluation and result interpretation.

Machine Learning - Gaurav Bhagat, Atharv Barge

- Ensuring Security and authentication
- Addition and Deletion of various nodes.
- Proving integration between Data and Blockchain Systems

Document Specialist - Advait Naik, Atharv Barge, Gaurav Bhagat, Om Acharya

- Preparation of project documentation.
- Creation of user manuals and project reports.
- Data visualization and reporting.

3.5.10 Project Plan 3.0

Introduction

Project Plan 3.0 represents a comprehensive overhaul of our project strategy for "Decentralized Aggregation Mechanism for Training Machine Learning models using Smart Contract System." This updated iteration is the culmination of our collective learning, integrating valuable insights gained from previous stages, and adapting to the evolving landscape of our project. Through meticulous analysis of project progress and careful consideration of feedback, we have refined our timelines, milestones, and tasks to ensure alignment with our overarching objectives.

By embracing continuous improvement and leveraging our experiences, Project Plan 3.0 embodies our commitment to excellence and adaptability. We recognize the importance of incorporating lessons learned into our planning process, enabling us to make informed adjustments and enhancements. This iterative approach not only strengthens our project's foundation but also positions us to navigate potential challenges with resilience and agility.

Updated Timeline

The project timeline is adjusted based on the progress made since the initial plan. It is important to ensure that all tasks align with the project's current status.

Project Plan	
Topic	Schedule
Finalizing Back-end Architecture Design	Week1
Implementing Back-end	Week2
Blockchain Module development	Week3
Python Module development	Week4
Porting project to Linux	Week 5
Paper presenting and publication	Week6
Testing modules design	Week7
Blockchain Testing Module	Week8
ML Testing Module	Week9
Report and final work	Week10

Yable 3.1- Revised Project Timeline

3.5.11 Conclusion

As a team, we delved into cutting-edge technologies like Ethereum, Solidity, and blockchain. Our existing machine learning expertise expanded with the inclusion of federated learning techniques. Notably, we introduced a novel data standardization stage that we believe sets our work apart. Addressing challenges inherent to applying federated learning within a blockchain system, we optimized model size by adjusting a single configuration parameter—reducing it by half without compromising accuracy.

While our initial focus was on intrusion detection, the versatile platform we developed can easily serve other blockchain-based federated learning applications. Moreover, the modular architecture encourages fellow developers to create diverse blockchain backends for this project.

3.5.12 PERT Table

A Program Evaluation and Review Technique (PERT) table will be used to schedule and manage project activities, with dependencies and estimated durations

Task	Dependencies	Estimated Duration (Days)
Project Initiation	-	5
Feasibility Study	Data Collection	15
Technology Study		20
UI Implementation	Data Analysis	15
UI Implementation	Model Building	10
Survey Paper	Data Analysis	10

Table 3.z: PERT Table

3.5.13 PERT Diagram

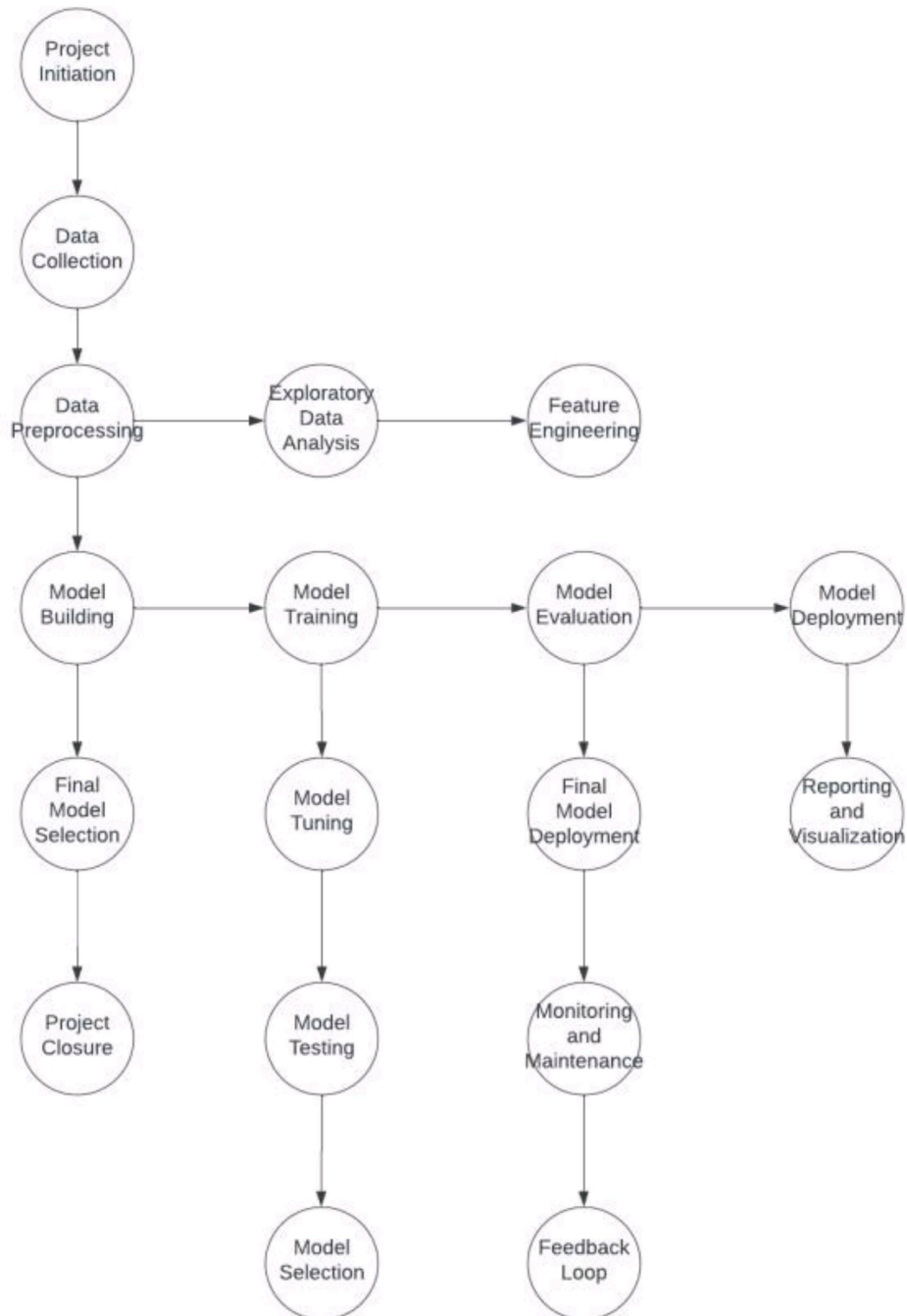
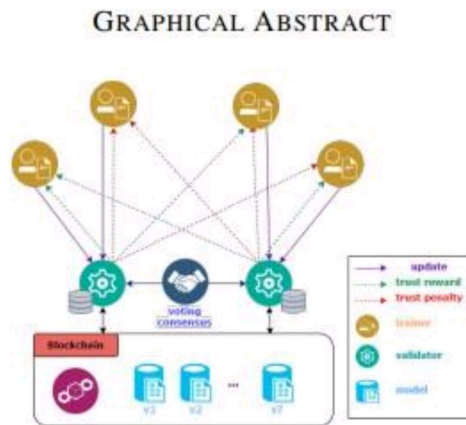


Figure 3.4: PERT Diagram Of Project

4. System Analysis and Design

4.1 System Architecture



Index Terms—Byzantine Fault-Tolerance, Federated Learning, Blockchain, Decentralized Systems, Distributed Machine Learning, Privacy Preserving

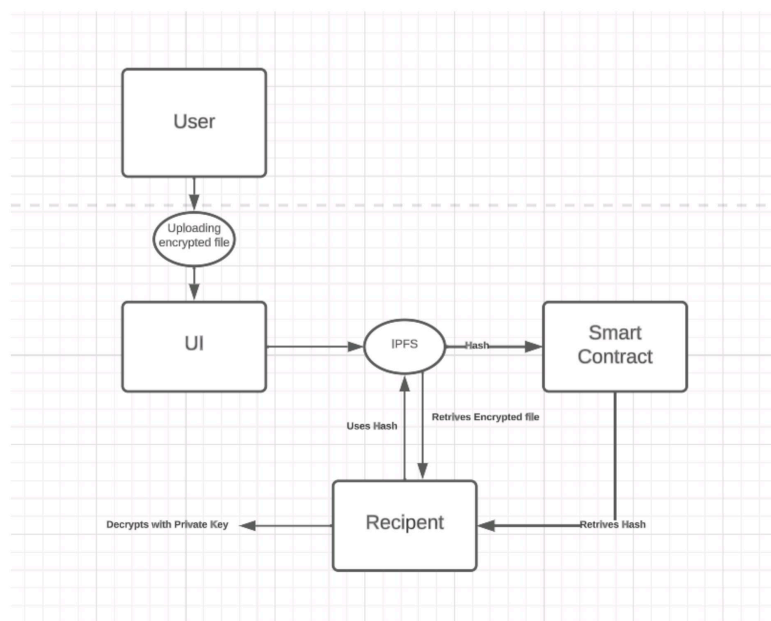


Figure 4.1: System Architecture

4.2 Necessary UML Diagrams

4.2.1 DFD

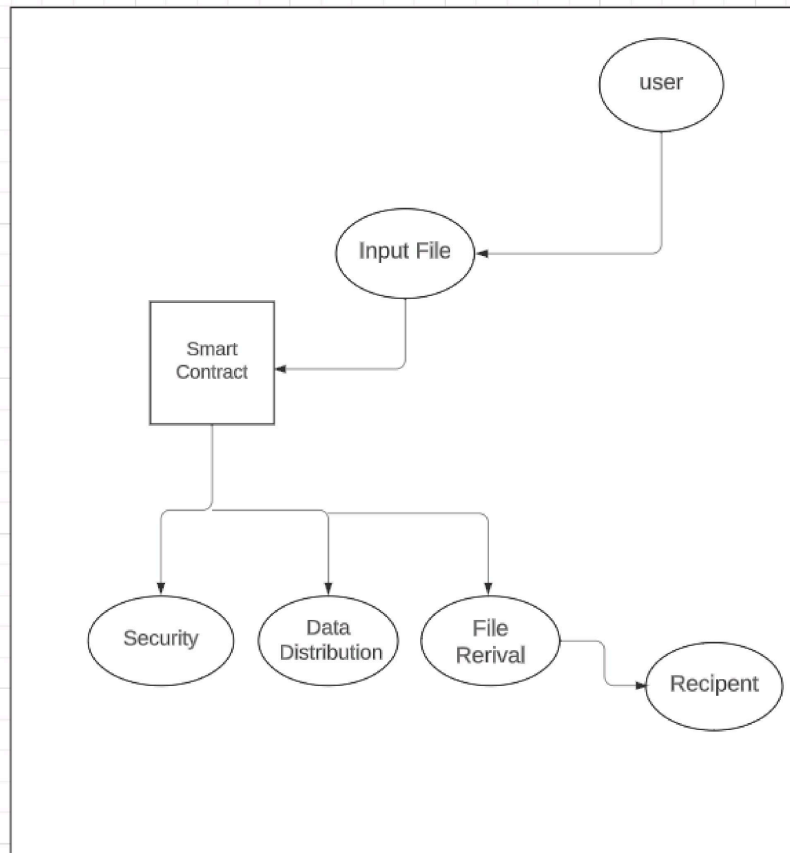


Figure 4.2: Data Flow Diagram

4.2.2 Activity Diagram

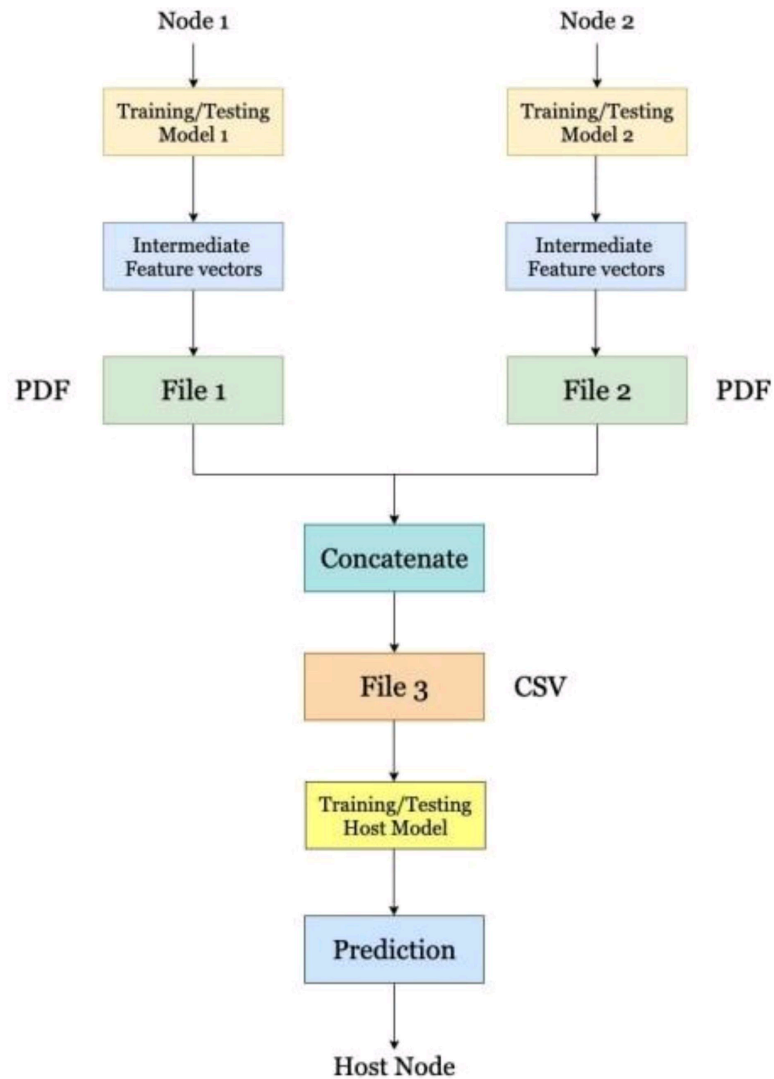


Figure 4.3: Activity Diagram

4.2.3 Sequence Diagram

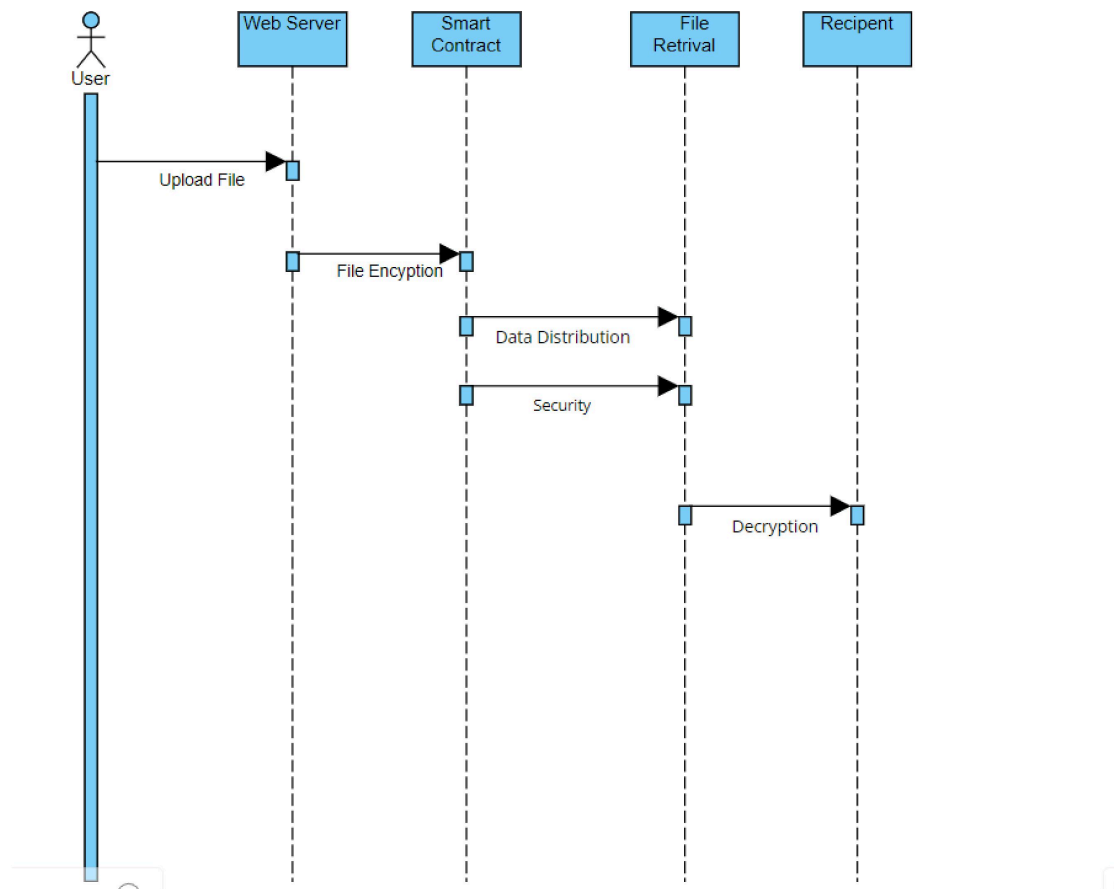


Figure 4.4: Sequence Diagram

4.3 Algorithm and Methodologies

- **Artificial Neural Network** - An artificial neural network, commonly known as a neural network, mimics the structure and functionality of the human brain. Comprising interconnected nodes or "neurons," these networks analyze and relay information. They excel at tackling intricate tasks such as image and speech recognition, leveraging data to learn patterns and make predictions. Renowned for their adaptability and ability to generalize, neural networks are extensively used in machine learning, natural language processing, and computer vision, playing a pivotal role in contemporary AI systems.
- **K Nearest Neighbors Algorithm** - The K-Nearest Neighbors (KNN) algorithm is a foundational supervised machine learning technique employed for both classification and regression tasks. At its core, KNN determines the class or value of a data point by analyzing the majority class or average value of its K nearest neighbors.

in a feature space. Despite its simplicity and intuitive nature, KNN's effectiveness relies on careful selection of the K value and may be influenced by the choice of distance metric. By leveraging proximity in high-dimensional spaces, KNN facilitates predictions based on the similarity between data points, rendering it invaluable in various data analysis and pattern recognition scenarios.

- **Random Forest** Random Forest stands as a potent ensemble learning technique within the realm of machine learning. Its mechanism involves constructing numerous decision trees during training and amalgamating their predictions to yield precise and resilient classifications or regression forecasts. Each decision tree is crafted using a random subset of the training data and features, fostering diversity and mitigating overfitting. Renowned for its versatility, Random Forest adeptly handles both categorical and numerical data, manages missing values, and gauges feature importance, rendering it a favored option across diverse domains like image classification and financial forecasting. Its ensemble nature and knack for diminishing variance while upholding predictive accuracy position it as a preferred solution for intricate machine learning endeavors.
- **Smart Contract** - A smart contract represents a self-executing and immutable digital agreement that operates on blockchain technology. These contracts are encoded with predetermined rules and conditions, automatically executing, enforcing, or facilitating transactions upon meeting specific criteria. By removing the requirement for intermediaries, smart contracts diminish fraud risks and guarantee transparency across diverse applications, including financial transactions, supply chain management, and decentralized applications (DApps). Their tamper-proof characteristics and autonomous execution capabilities present a promising innovation for enhancing efficiency and trustworthiness across various processes in a decentralized, secure, and cost-effective manner.
- **Synchronization-Distributed learning** relies on synchronized model updates to prevent parameter discrepancies and ensure model integrity. Synchronization barriers mark rounds where updates are shared, enhancing convergence and reducing model degradation. However, this comes at the cost of increased communication overhead. The process involves alternating between computation and communication phases. Several synchronization techniques are employed:

a. Bulk Synchronous Parallel (BSP):

BSP is a straightforward approach where computation and communication occur in

alternating rounds. During computation rounds, each node performs its computation independently. Once computation is complete, all nodes synchronize and share their updates to create a new model. The main advantage of BSP is its consistency, ensuring that all nodes have the same version of the model. However, BSP suffers from the bottleneck of requiring all nodes to finish their computation before proceeding to the next round of updates. This waiting time can lead to inefficiencies, especially if some nodes finish their computations significantly earlier than others.

b.5tale Synchronous Parallel (SSP):

SSP is similar to BSP but introduces more relaxed synchronization barriers. Instead of waiting for all nodes to finish computation, SAP allows a certain number of iterations beyond the preset limit. Once this leeway is reached, all nodes pause until the next synchronization barrier. SSP strikes a balance between BSP's strict synchronization and fully asynchronous approaches. While SSP offers flexibility and can accommodate varying computation times among nodes, convergence may degrade with increased staleness, i.e., the time gap between updates from different nodes.

c. Barrierless Asynchronous Parallel (BAP):

BAP eliminates synchronization barriers altogether, allowing nodes to communicate asynchronously. This approach significantly reduces communication overhead and can lead to higher speedups, especially in scenarios where nodes have varying computation times. In BAP, nodes can continue training until a new model is released. However, this lack of synchronization may lead to slower convergence and potential inaccuracies, as nodes may not be working with the most up-to-date model parameters.

d. Approximate Synchronous Parallel (ASP):

ASP focuses on limiting parameter accuracy instead of staleness to reduce synchronization costs. Instead of waiting for all updates, ASP may ignore updates that are deemed insignificant. This approach aims to strike a balance between synchronization efficiency and model accuracy. However, implementing ASP can be challenging as determining which updates are insignificant requires careful consideration, and it may introduce additional complexity to the system.

5. Implementation

5.1 Stages of Implementation

511 Data Preprocessing

- Data preprocessing aims to mitigate inconsistencies in the data we handle. This encompasses tasks such as data cleaning, null value removal, standardization, DataFrame fitting, and similar processes.
- The datasets chosen for this paper must adhere to specific criteria, as our ultimate goal is federated learning. It is essential that the dataset can be divided into private and public features, which will subsequently undergo separate training processes.
- We partition the dataset into two segments: private data, which undergoes separate training on a secure system where it resides, and public data, which can be transferred with lower risk of exposure.

512 Implementation of Modules

- This module encompasses training two distinct deep learning models on individual nodes. These models are tailored to the available data, with one node containing personal details and the other containing bank-specific information.
- Following model training, the extraction of feature representations from the third hidden layer of each model is necessary. These feature representations play a crucial role in subsequent processing steps.
- Feature Representation Extraction: The acquired feature representations for the complete training dataset must be stored in Portable Document Format (PDF) on both nodes.
- Data Storage on Nodes: The Encryption Module is tasked with encrypting the files using the host's public key, ensuring that only the host can access and decrypt the files using their private key.
- Data Storage on Nodes: The integration of IPFS into the system facilitates file storage and retrieval, enabling the storing and exchanging of data in a distributed setup.

- **Smart Contract Development:** This module entails the development and deployment of a smart contract on the Ethereum Blockchain. The smart contract is responsible for storing the hash of the files and governs the execution of transactions related to data retrieval. It ensures transparency and trackability in the system.
- **Data Retrieval by the Host:** This module involves implementing the necessary functions for the host to retrieve files using the hash stored in the smart contract.
- **User Interface for File Upload:** The development of a user-friendly interface for file upload is essential for individual nodes to upload the encrypted files to IPFS.

5.2 Experimentation Setup

1. **Dataset Preparation:** Choose an appropriate dataset pertinent to the research topic (such as another banking or financial dataset for loan prediction). Preprocess the dataset by cleaning, normalizing, and partitioning it into training and testing sets. Implement vertical partitioning of the data to mirror the decentralized data setup outlined in the paper.
2. **Model Architectures:** Specify the neural network architectures for both the local models and the host model, such as feed-forward neural networks (FFNN). Define the number of layers, the neurons in each layer, and the activation functions for each model. Ensure that the architecture is tailored to the characteristics of the data partitions, including personal details and bank-specific details.
3. **Hyperparameters:** Establish the hyperparameters for training, encompassing variables such as learning rate, batch size, and the number of epochs. Select the appropriate loss function for training, such as binary cross-entropy, and determine the optimization algorithm, such as Adam.
4. **Decentralized Training Setup:** Implement the mechanism for decentralized training and feature aggregation using blockchain and IPFS. Guarantee that the system securely stores, retrieves, and exchanges intermediate feature vectors while maintaining data privacy.
5. **Training:** Train the local models on their respective data partitions, capturing intermediate feature representations from the third hidden layer for each model.
6. **Feature Aggregation:** Aggregate feature representations from the local models to construct a unified dataset, which will serve as the input for training the host.

6. Results

6.1 Results of Experiments

We utilized a dataset comprising 25,192 labeled samples, featuring 41 attributes, with 3 being categorical. The dataset encompasses two classes: anomaly and normal. Initially, we partitioned the dataset into training and validation sets using an 80

In our initial experimentation, we employed a single hidden layer with 50 neurons and ReLU activation function. Utilizing $C = dpre = 1$, conducting 10 global epochs, 5 local epochs, and employing a batch size of 32, we achieved a maximum validation accuracy of 99.17

Across most of our experiments, modifying the precision level of the model (16-bit, 32-bit, or 64-bit floats) on the blockchain yielded negligible differences. It's noteworthy that this adjustment solely impacts how the model is communicated, not its internal representation, which consistently employs 64-bit floats. Specifically, clients receive the global model from the blockchain using 16-bit floats, convert these values to 64-bit floats for gradient descent operations, and subsequently report updates back to the blockchain using 16-bit floats.

6.2 Result Analysis

6.2.1 Experiments

We noticed that altering the hidden layers had minimal effect on the ultimate outcome. After gradually simplifying the model, we ultimately opted for a single linear layer represented as $Ax + b$ with 2 neurons for output. Despite the excessive simplification, we achieved an accuracy of 97.28

However, data standardization proved indispensable in this dataset as accuracy plummeted to 50% without it. We tried different learning rates and hyperparameters but to no avail. Furthermore, using 16-bit floats lead to NaNs without data standardization.

In Figure 6.1, we reduced the learning rate to 0.0001, increased the global epochs to 16, and experimented with different α parameters while keeping the rest constant. We see that even if only one client shares y and o_z , the accuracy we attain is almost as high

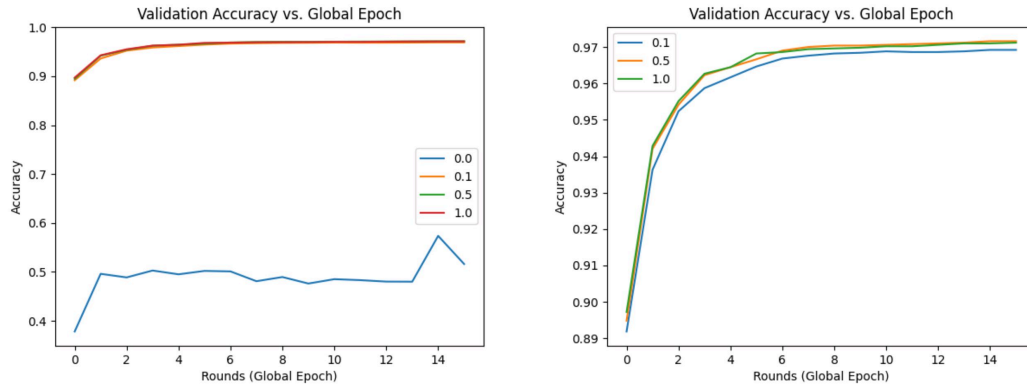
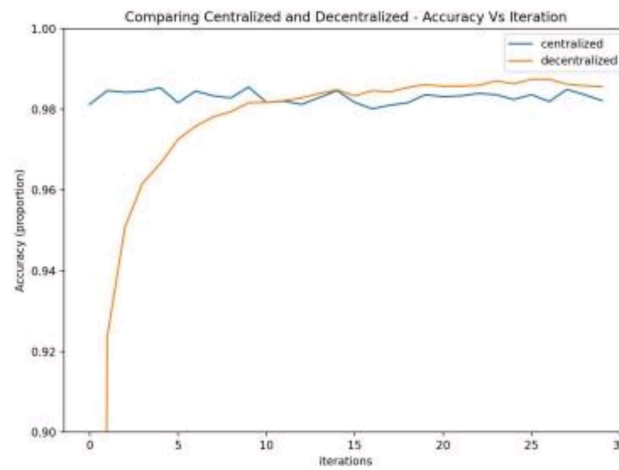


Figure 6.1: Accuracy vs. Epoch Plots with Varying C_{pre}



as the case in which all clients share standardization data. However, we don't expect this to be the case when the data is non-IID, which is usually the case in real life.

we fixed C_{pre} to 0.5 and experimented with different C values. However, the differences were negligible.

Finally, we tested the model on 10% of the larger KDD Cup 1999 Data (?), featuring 23 class labels. Despite the increased size and complexity of the dataset, we still got 98% accuracy on the validation set with the single layer linear model, as long as the data standardization step is in-place.

6.3 Testing

White Box

Unit Testing

Model Training Function : Verified that the model training function correctly trains the model with specified parameters (e.g., hidden layers, neurons, activation function, optimization parameters). Also ensured that the function produces the expected maximum accuracy on the validation set (e.g., 99.17%).

Data Standardization Process : Tested the data standardization function to ensure it maintains accuracy, preventing it from plummeting to 50% without standardization. Verified that different precision levels (16-bit, 32-bit, 64-bit floats) don't lead to NaNs when data standardization is not applied.

Gradient Descent Function : Validated the gradient descent function to ensure it updates model parameters correctly based on calculated gradients. Tested different learning rates and hyper parameters to optimize model performance, despite no significant improvements observed.

Model Evaluation Function : Verified that the model evaluation function accurately assesses model performance on various datasets. Ensured the single-layer linear model achieves 98% accuracy on the validation set, provided data standardization is applied.

The system underwent rigorous unit testing to validate individual components' functionality independently. This ensured that each module performs as expected within the federated learning framework.

Integration Testing

Integration testing was conducted to examine the interaction between different modules or components of the system. This aimed to verify the seamless integration of various parts and their collective behavior.

Test Cases

A comprehensive suite of test cases was designed to evaluate the system's functionality from an external perspective, without considering its internal structure. These test cases covered various scenarios and inputs to ensure the system behaves correctly under different conditions. These test cases helped overcome limitations of blockchain technology.

In order to inspect the limitations imposed by the blockchain system, We created a simple testing script that checks the gas fee associated with varying model sizes. In Table , gas fee required to perform 1 global and 10 local updates (roughly 1 round) on test blockchain setup is given for several byte sizes. Third column gives the ratio of the current gas fee to the gas fee for 1 byte.

In the test system, all accounts start with 1000000 ETH, which is roughly equivalent to 1.2 billion dollars at the time of writing. After 50000 byte size, we cannot do even 1 round of updates on the blockchain with this amount.

Byte Size	Gas Fee (Gwei)	Ratio to Base Gas Fee
1	736795	1.0000
10	7379g3	i.0016
100	841886	1.1426
1000	1629055	2.2110
10000	10457631	14.1934
20000	20295764	27.5460
30000	30004373	40.7228
40000	39810720	54.0323
50000	Out of gas	-

Table 6.1: Byte Size vs. Gas Fee

Therefore, our model size must be below 40000 in bytes, and around 10000 if we want to be financially reasonable. For example, a model with 100 inputs (features), 50 neurons in a single hidden layer, and JO outputs (classes) require $100 \times 50 + 50 \times 10 = 5500$ weights and $50 + 10 = 60$ biases, resulting in 5560 floating point numbers in total. If we use 16, 32, or 64-bit floats, model's byte size will be 11120, 22240, and 44480 respectively.

Gas fees for communicating means and standard deviations tend to be a lot less since it's just 1D data with its length equal to the number of features.

Summary of Black Box Testing

Summary of the black box testing results was compiled, outlining the overall performance of the system based on the conducted tests. This summary provided insights into any detected issues, areas of improvement, and the system's compliance with specified requirements.

7. Conclusion and Future Scope

7.1 Conclusion

During this project, We learned a lot about new technologies such as Federated Learning, Ethereum, Solidity, and blockchain in general. We also tackled the problems that arise when federated learning is applied in a blockchain system, such as model size restrictions. By changing a single configuration parameter, the model size on the blockchain can be halved without any significant impact on accuracy. The project we developed can be utilized for other blockchain-based federated learning applications easily.

By leveraging a smart contract system and blockchain technology, we have successfully demonstrated a mechanism that allows for the training of deep learning models on distributed data while ensuring data privacy. Our approach, which focuses on aggregating intermediate representations from local Artificial Neural Network (ANN) models, is particularly valuable in scenarios where data samples are distributed across multiple nodes. The results from this phase of the project consistently show that our approach outperforms individual node training and can compete with centralized data setups. This not only validates the effectiveness of our method but also highlights its practical significance for organizations seeking to balance data privacy with the benefits of data sharing in the realm of deep learning.

7.2 Limitations of the Project

- ' **Data Compatibility:** The success of our approach relies on data being distributed across multiple nodes while maintaining the same number of data samples. In scenarios where data samples differ significantly in size or structure, the method may not be as effective.
- ' **Smart Contract Overhead:** Implementing and managing smart contracts can introduce additional computational overhead. This may result in increased processing time and resource consumption, especially in scenarios with a large number of

participating nodes.

- **Data Security:** While our approach focuses on data privacy, it does not provide an absolute guarantee against data breaches, as the blockchain itself is not immune to vulnerabilities. It might be required to take additional steps to guarantee complete data security.
- **Communication Overhead:** The need for nodes to communicate and share intermediate representations introduces communication overhead. In cases where nodes are geographically distant, this overhead could impact the efficiency of the method.

7.3 Future Scope

The future scope of this project is not only promising but also versatile, offering applications across a spectrum of domains such as healthcare, finance, e-commerce, and more. As data privacy remains a paramount concern in these fields, enhancing the project's data security measures is of utmost importance. This entails the integration of advanced encryption techniques and privacy-preserving algorithms, ensuring that sensitive data remains impervious to breaches.

Addressing the issue of scalability is another vital aspect of the project's future expansion. As its adoption grows and the demand for larger datasets and more participants increases, it becomes crucial to optimize the blockchain's performance. Techniques like sharding and layer-2 solutions can mitigate computational overhead and reduce transaction processing times, making the approach practical for large-scale applications.

Moreover, interoperability is a cornerstone of the project's future development. Ensuring that the methodology seamlessly integrates with various blockchain platforms and smart contract systems not only eases adoption but also fosters a standardized framework for secure, distributed data training in deep learning applications. This compatibility facilitates its use across different organizations and industries, promoting a unified approach to data privacy and sharing.

the project's future scope is marked by its adaptability, fortified data security measures, scalability solutions, and enhanced interoperability. As it advances, it has the potential to

redefine how data privacy and sharing are managed in deep learning applications across a multitude of sectors, ultimately contributing to the advancement of secure, distributed data training.

REFERENCES LIST:

1. H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS), Fort Lauderdale, FL, USA, 2017, vol. 54, pp. 1273–1282.
2. J. Konečný, H. B. McMahan, and D. Ramage, "Federated Optimization: Distributed Optimization Beyond the Datacenter," arXiv preprint arXiv:1511.03575, Nov. 2015.
3. R. Shokri and V. Shmatikov, "Privacy-Preserving Deep Learning," in Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur., Denver, CO, USA, 2015, pp. 1310–1321.
4. J. Xu, H. Zhu, Y. Jin, and S. Liu, "Federated Learning with Differential Privacy: Algorithms and Performance Analysis," in Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP), Toronto, ON, Canada, 2021, pp. 5860–5864.
5. H. Eichner, K. Bonawitz, and W. Grieskamp, "Federated Learning: Strategies for Improving Communication Efficiency," in Proc. NeurIPS Workshop Federated Learn. Priv. Preserving Mach. Learn., Vancouver, BC, Canada, 2019.
6. Y. Zhang and P. Mahassel, "Secure Federated Learning for Neural Networks," in Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS), Vancouver, BC, Canada, 2020.
7. K. Ota and S. Ichikawa, "Decentralized Aggregation Mechanism for Training Machine Learning Models Using Smart Contracts," in Proc. IEEE Int. Conf. Blockchain (Blockchain), Atlanta, GA, USA, 2020, pp. 1–6.
8. R. Bellazzi, N. L. Bragazzi, and M. Favato, "Systematic Review Techniques for Assessing Prior Research Quality," J. Biomed. Inform., vol. 58, pp. 1–9, Oct. 2015.
9. S. Tschitschek, S. Song, A. Krause, and A. Veith, "Learning Determinantal Point Processes in Sublinear Time," in Proc. 34th Int. Conf. Mach. Learn. (ICML), Sydney, NSW, Australia, 2017, pp. 3921–3930.

10. R. Opdenakker and H. P. Wouda, "Blockchain-Based Transaction Authentication Systems," in Proc. 12th Int. Conf. E-Business (ICE-B), Lisbon, Portugal, 2015, pp. 1–6.
11. H. Zhu, Y. R. Shayan, and Y. Xia, "Blockchain in IoT: Ownership, Integration, and Record Keeping," in Proc. IEEE Int. Conf. Internet Things (iThings), Chengdu, China, 2018, pp. 1–6.
12. Deloitte, "Rebuilding the Blockchain 'Trust Machine'," Wall Street Journal, Mar. 11, 2024.